

Curso de Especialización Ciberseguridad en las TIC

Ejercicios vulnerabilidades

Objetivos

- Detectar los problemas de los códigos.
- Identificar los riesgos.
- Buscar una solución a las vulnerabilidades.
- Probar que funciona correctamente sin vulnerabilidades.

Resumen

Autora: Marina del Pilar López Oliva

Octubre 2025

IES Camp de Morvedre

Obra publicada con [Licencia Creative Commons Reconocimiento Compartir igual 4.0](https://creativecommons.org/licenses/by-sa/4.0/)



Revisión

Revisión	Fecha	Descripción
1.0	28/10/2025	Realización de los ejercicios hasta el 5.
1.1	29/10/2025	Realización de los ejercicios del 6 al final.

Índice de Contenidos

Ejercicio 1 - Ejercicio resuelto.....	4
Ejercicio 2 - Cross-Site Scripting (XSS) en salida HTML.....	4
Ejercicio 3 - Almacenamiento inseguro en móvil.....	5
Ejercicio 4 - Deserialización insegura.....	5
Ejercicio 5 - Exposición de datos en logs.....	6
Ejercicio 6 - Control de acceso insuficiente (IDOR).....	6
Ejercicio 7 - Uso de cifrado débil / hashing inadecuado.....	6
Ejercicio 8 - Subida de ficheros sin validación.....	6
Ejercicio 9 - Exposición de APIs sin autenticación.....	6
Ejercicio 10 - Falta de validación en parámetros que causa lógica rota.....	6
Webgrafía.....	7

Ejercicio 1 - Ejercicio resuelto

El problema está en la concatenación de username en SQL. Se detecta mediante una prueba con pytest. La prueba envía una cadena que, si es concatenada en la consulta, convierte la condición en verdadera y devuelve filas no deseadas. Si result no es None, hay vulnerabilidad.

El riesgo es la entrada controlada por el usuario, que puede manipular la consulta.

La solución es usar consultas parametrizadas, en sqlite3. Los parámetros no se interpretan como SQL.

Por lo tanto, si volvemos a hacer la prueba, debe devolver ahora None para la cadena maliciosa.

Ejercicio 2 - Cross-Site Scripting (XSS) en salida HTML

El problema reside en un Cross-Site Scripting. La línea vulnerable es: `out.println("<h1>Hola " + name + "</h1>");`. Este ataque consiste en la introducción de código malicioso desde otros sitios y que es capaz de ejecutarse en el cliente.

Es la introducción de código en la página web atacada para que, cuando un usuario legítimo lo use, el atacante pueda robar su información o incluso escalar privilegios. Se puede realizar de diversas maneras. En este caso, al introducir el nombre y publicarlo tal cual, se cargará en su contenido un script de un sitio malicioso. Esto daría como resultado la posibilidad de ejecutar código de cualquier tipo en el navegador web y pondría al descubierto cualquier tipo de interacción del usuario con la página atacada.

Esto puede provocar robo de información sensible de los usuarios, suplantación de identidad y secuestros de sesiones, y redirección a sitios maliciosos o phishing.

Para evitar el ataque se podría validar y escapar correctamente los datos de entrada. Escapar caracteres especiales (<, >, ', ") evita que se interpreten como código HTML o JavaScript. Es decir, podemos cambiar la línea por `out.println("<h1>Hola " + escapeHTML(name) + "</h1>");`. Y añadir al código:

```
private String escapeHTML(String input) { return input.replace("&", "$amp;")  
                                                    .replace("<", "&lt;")  
                                                    .replace(">", "&gt;"); }
```

```
        .replace("\"", "&quot;")  
        .replace("'", "&#x27;");  
    }  
}
```

Ejercicio 3 - Almacenamiento inseguro en móvil

El problema en este caso reside en la línea *f.write(token)*. Esto nos indica que el token se almacena en texto plano sin protección. No se llega a cifrar de ninguna forma. Esto deja al token vulnerable.

Los riesgos que puede tener dejar el código así son variados. Cualquier aplicación con permisos de almacenamiento puede leer el token y tener acceso físico al dispositivo. Además, pueden escalar privilegios, aplicaciones maliciosas pueden robar el token para suplantar la identidad. Y sin cifrado, el token es legible para el atacante inmediatamente. No hay protección contra extracción.

Podemos solucionar este caso en cifrado con librerías seguras. También podríamos buscar la manera de cifrar el token de manera segura dentro del código. O utilizar librerías como keyrings o el KeyStore.

Ejercicio 4 - Deserialización insegura

En este caso el problema está en la línea: *return ois.readObject();* teniendo el *ObjectInputStream* insegura. Esto nos devuelve una deserialización insegura. La deserialización insegura es el proceso de tomar datos estructurados y convertirlos de nuevo en objetos en memoria.

El riesgo surge cuando se aplica la deserialización a datos no confiables sin validación. Un atacante puede manipular la entrada para activar una cadena de gadgets, rutas de código existentes utilizadas de forma no deseada. Esto puede provocar ejecución de código remota.

Esto se puede detectar mediante pruebas de seguridad de aplicaciones estáticas. Dentro de esta se realizan pruebas unitarias con entradas maliciosas simuladas. Con esto, podemos comprobar si se está realizando la deserialización insegura.

Para solucionar esta vulnerabilidad se puede tener límites de confianza para que deserialice únicamente desde fuentes autenticadas y verificadas. También pueden usarse API seguras como bibliotecas seguras con validación. Además se pueden restringir los tipos de objetos permitidos aplicando esquemas JSON. O

agregar una desrealización, haciendo comprobaciones de seguridad para plantillas de revisión de relaciones públicas.

Ejercicio 5 - Exposición de datos en logs

La vulnerabilidad en este código reside en `logger.info(f'Attempt login username={username} password={password}')`. Esta línea procesa una autenticación mediante logs, manteniendo la contraseña en ellos. Esto presenta riesgos significativos para la seguridad, ya que cualquier persona con acceso a los logs, puede identificar fácilmente las credenciales. En consecuencia, pueden obtener acceso a sistemas protegidos, bases de datos y otros recursos sensibles.

Como en los otros ejercicios, esta vulnerabilidad puede ser corregida si encriptamos la contraseña. En vez de validar las credenciales en logs con el texto plano, podemos utilizar los hashes para mantenerlo más difícil de identificar. Otra técnica sería hacer la autenticación de forma que no sea visible fácilmente, como lo es mediante logs.

Se puede usar un almacenamiento seguro de credenciales, como el almacenamiento cifrado en bases de datos. Esto proporciona una capa adicional de protección. Lo que dificulta el acceso no autorizado a las credenciales, incluso si se accede al código fuente.

También podemos usar la gestión de claves y contraseñas. Implementar una sólida gestión de claves y contraseñas es crucial.

Ejercicio 6 - Control de acceso insuficiente (IDOR)

El problema está en cómo devuelve el documento cuando se pide. La línea `"return self.docs.get(doc_id)"` sólo devuelve el documento si existe, sin comprobar el propietario. Este fallo se denomina "IDOR", Identificador Directo a Objetos no Protegidos. El programador deja un vínculo expuesto a un componente interno de un software. La amenaza más prominente de este fallo es la posibilidad de que un pirata informático pueda visualizar el contenido confidencial de otro usuario.

Para evitar este problema tienen que ponerse controladores de entrada adecuados en los softwares. En este caso, estaría bien comprobar primero quién lo ha pedido y si tiene acceso al documento. Si tiene acceso al documento, entonces mostrarlo. Por el contrario, si no tiene acceso, mostrar un aviso, no el interior del documento. Se debería usar un identificador de sesión único, y

mediante ese identificador, ver si el documento que le pide al programa se puede mostrar o no, dependiendo de los permisos.

Ejercicio 7 - Uso de cifrado débil / hashing inadecuado

La vulnerabilidad en este caso es el uso de cifrado débil. Utiliza MD5, un algoritmo de hashing obsoleto, comúnmente representado como una cadena de 32 caracteres hexadecimales.. Se considera inseguro para propósitos criptográficos debido a vulnerabilidades conocidas.

El uso continuo de algoritmos de cifrado débiles pone los datos confidenciales en riesgo. MD5 pasó de ser un algoritmo de hash seguro e irrompible, a un algoritmo de hash débil y roto.

Para solucionar este problema, se podrían usar algoritmos de hash compatibles y fuertes como SHA-2.

Ejercicio 8 - Subida de ficheros sin validación

La línea del código `"path = os.path.join(self.upload_dir, filename)"` es vulnerable a path traversal. Path traversal es una vulnerabilidad de seguridad que permite a un atacante acceder a archivos y directorios arbitrarios en un sistema mediante la manipulación de rutas de archivos. Esto puede conducir a un acceso no autorizado a archivos confidenciales.

Si una aplicación web toma la entrada del usuario para construir una ruta de archivo y no la desinfecta correctamente, un atacante puede manipular la entrada para atravesar directorios y acceder a archivos sensibles.

Para asegurar su aplicación, debe validar y desinfectar la entrada del usuario.

- Se puede normalizar la ruta y restringir el acceso mediante el uso de `"os.path.realpath()"`. Así obtenemos la ruta absoluta y comprobamos si se mantiene dentro del directorio previsto.
- Podemos utilizar la lista blanca que solo permite nombres de archivos específicos y seguros.
- También es posible utilizar Python's `pathlib`, que proporciona un mejor manejo de la ruta.
- Otras soluciones pueden ser la implementación de controles de acceso al lado del servidor, la deshabilitación de los listados de directorios, o la utilización de almacenamiento seguro de archivos.

Ejercicio 9 - Exposición de APIs sin autenticación

Al utilizar “return userService.findAll();” sin nada más, el código nos devuelve los usuarios sin autenticar. La autenticación es fundamental para garantizar la seguridad y la integridad de los usuarios. Este proceso es esencial para proteger la información sensible de accesos no autorizados, y para asegurar que cada usuario tenga un nivel de acceso adecuado a los recursos disponibles basado en sus permisos.

Este problema se puede solucionar mediante la autenticación por API Key. Consiste en una clave secreta que se envía junto a la solicitud HTTP. Aunque es fácil de implementar, su principal desventaja es la falta de flexibilidad.

También sería una opción utilizar la autenticación básica. Una técnica sencilla donde el usuario y la contraseña se codifican en base64 y se envían en el encabezado de autorización de cada solicitud HTTP.

Ejercicio 10 - Falta de validación en parámetros que causa lógica rota

En este código tenemos un problema de falta de validación. En la función “def transfer()” no se validan los límites, ni tampoco se comprueba que “from_acct” tenga saldo suficiente para la acción.

Al tener una falta de validación en los parámetros, causamos una vulnerabilidad llamada “Business Logic Flaw”. Esto se produce cuando la funcionalidad legítima de una aplicación se utiliza indebidamente de maneras que los desarrolladores nunca tuvieron la intención. Estas son debilidades a nivel de diseño, no errores de implementación. El código puede no ser inseguro pero el diseño permite a los usuarios hacer cosas que el negocio nunca tuvo la intención.

En este caso se produce por una falta de validación. Las entradas insuficientes interrumpen los flujos de trabajo. Habría que establecer los límites necesarios en el código además de comprobar si “from_acct” tiene o no saldo.

La prevención de fallos en la lógica empresarial comienza con un diseño acertado de la lógica empresarial y una implementación minuciosa. Los programadores necesitan comprender de manera total la lógica empresarial que están instaurando y tener en consideración todas las posibles formas en que esta podría ser mal utilizada.

Webgrafía

<https://www.arsys.es/blog/ataques-xss-que-son-y-como-evitarlos>

<https://www.bleepingcomputer.com/news/security/microsoft-teams-stores-auth-tokens-as-clear-text-in-windows-linux-macs/>

<https://developer.android.com/reference/java/security/KeyStore>

<https://xygeni.io/es/blog/how-insecure-deserialization-becomes-remote-code-execution/>

<https://blog.hackmetrix.com/insecure-deserialization/>

<https://blog.hackmetrix.com/hard-coded-credentials/>

<https://lab.wallarm.com/what/vulnerabilidad-idor-referencias-directas-a-objetos-inseguras/?language=es>

<https://docs.digicert.com/es/certcentral/certificate-tools/discovery-user-guide/tls-ssl-certificate-vulnerabilities/weak-hashing-algorithm.html>

<https://osintteam.blog/path-traversal-and-remediation-in-python-0b6e126b4746>

<https://www.palentino.es/blog/mecanismos-de-autenticacion-para-apis-desde-clasicos-hasta-los-mas-modernos/>

<https://infosecwriteups.com/business-logic-flaws-a-bug-hunters-handbook-293f6a89a7f4>